SMART CONTRACT SECURITY AUDIT

rwa-tk
Internal Security Review

RWA Tokenization Platform — Audit Report

Manual review, static analysis (Slither), Foundry unit / fuzz / invariant tests, governance lifecycle simulation, and deployment hand-off verification. Scope: 9 first-party Solidity contracts, 3 deployment scripts, subgraph mappings, frontend access pattern.

PROJECT
Blockchain Technologies 2 — Option C

REPOSITORY
Blockchain2_FINAL_Alikhan_Zarina

COMMIT HASH
`8ac3c73`

DOCUMENT TYPE
Security Audit Report

SOLIDITY VERSION
0.8.20 · via_ir · optimizer 200 runs

TARGET CHAIN
Base Sepolia

HIGHEST SEVERITY
Low (production code)

AUTHORS
Alikhan · Zarina

Table of Contents

1. Executive Summary

This report documents the internal security review of the **Option C — RWA Tokenization Platform**, a reserve-backed asset-tokenization protocol composed of an upgradeable asset registry, ERC-4626 reserve vault, ERC-20 asset token with role-gated mint/burn, a constant-product AMM, a Chainlink-compatible price-oracle wrapper, and an OpenZeppelin Governor stack secured behind a 2-day TimelockController.

The review covered manual code reading of every first-party Solidity file (9 contracts, ~750 source lines excluding mocks), static analysis with Slither, dynamic analysis through Foundry's unit / fuzz / invariant harness (**82 passing tests**, **96.64 % line coverage**, 7 active invariants), and inspection of the deployment + post-deploy verification scripts.

1.1 Summary of findings

Severity	Production code	Test & sample code	Off-chain	Total
CRITICAL	0	0	0	0
HIGH	0	2 (case-studies)	0	2
MEDIUM	0	0	0	0
LOW	4	0	1	5
INFORMATIONAL	6	—	—	6
GAS	2	—	—	2
Total	**12**	**2**	**1**	**15**

> **Overall assessment.** The production code contains **no critical, high, or medium-severity issues.** All identified production-code findings are Low or Informational and relate to hardening opportunities (proxy initialization, governance distribution, slippage defaults), not to exploitable economic or authorisation flaws. The two High-severity findings reproduced in the test suite are deliberate case studies (vulnerable / hardened pairs) used to demonstrate reentrancy and access-control attack patterns; they do not exist in any production contract.

1.2 Key strengths

- Deployment script renounces **every** deployer role and transfers admin to the Timelock in the same broadcast; `VerifyPostDeploy.s.sol` mechanically asserts the hand-off with 14 conditions.
- All four ERC-4626 entry-points (`deposit` , `mint` , `withdraw` , `redeem`) are protected by `nonReentrant` and follow checks-effects-interactions.
- AMM swaps require an explicit min-output argument via the protected overload; the frontend always uses it.
- Oracle wrapper performs four independent freshness/validity checks before returning a price.

- Vault accounting is conserved by seven property-based invariants that run 32 × 64 calls per `forge test` invocation.
- 96.64 % line coverage and 92.11 % function coverage on `src/` per the repository's `forge coverage` output.

1.3 Areas for hardening

1. Replace the custom `bool _initialized` guard with OpenZeppelin's `Initializable` and invoke `_disableInitializers()` in the implementation constructor (finding L-01).
2. Add a `__gap` array to `AssetRegistry` for safer future upgrades (finding L-02).
3. Correct the subgraph's `handleVoteCast` state mutation, which incorrectly overwrites `Proposal.state` to `"Active"` on every for/abstain vote (finding L-03).
4. Document or reduce the deployer's bootstrap GovernanceToken concentration (1 M of 100 M cap — 1 % supply but 25 % of quorum on day-one) (finding L-05).

2. Audit Scope

2.1 In-scope source files

Path	Lines	Role
`src/AMM.sol`	143	Constant-product market
`src/AssetReceipt.sol`	54	ERC-1155 receipt layer
`src/AssetRegistry.sol`	118	UUPS-upgradeable issuer / asset registry
`src/AssetRegistryV2.sol`	16	V2 implementation (append-only)
`src/AssetToken.sol`	54	ERC-20 protocol token with role gates
`src/AssetVault.sol`	125	ERC-4626 reserve vault
`src/Governance.sol`	120	GovernanceToken + ProtocolGovernor
`src/PriceOracle.sol`	39	Chainlink AggregatorV3 wrapper
`src/VaultFactory.sol`	33	CREATE / CREATE2 vault deployer
`script/Deploy.s.sol`	126	Sepolia deployment + privilege hand-off
`script/VerifyPostDeploy.s.sol`	65	Post-deploy assertion harness
`script/CreateProposal.s.sol`	80	Sepolia demo proposal script
`subgraph/src/protocol.ts`	174	Event handlers (AssemblyScript)
`subgraph/subgraph.yaml`	138	Subgraph manifest (4 data sources)
`subgraph/schema.graphql`	56	Subgraph schema (5 entities)

2.2 Out of scope

- **OpenZeppelin Contracts** (v4.x) internals — treated as a trusted dependency.
- **Foundry / forge-std** internals — test-only helpers.
- The flattened source files (`src/flat_*.sol` , `script/flat_*.sol`) — these are duplicates of the canonical source.
- The React frontend (`frontend/`) — reviewed only as a consumer of the on-chain interface to validate that the canonical entry-points are called correctly; full JavaScript-level audit is outside scope.
- Off-chain key management, RPC providers, indexer hosting.
- Slide deck and presentation materials.

2.3 Commit information

The review covers the repository at the snapshot indicated on the cover page. The repository's `README.md` reports the in-tree state of **82 passing Foundry tests**, **0 unresolved Slither findings**, and the governance / hand-off architecture described in §6.

3. Methodology

The review combined manual code reading, static analysis, and exhaustive dynamic testing. Each finding was reproduced (where applicable) against the in-repository test harness or against a written PoC.

3.1 Activities

Activity	Tooling	Outcome
Source code reading	Manual (every file in `src/` and `script/` read line-by-line)	Findings L-01 … L-04, I-01 … I-06, G-01, G-02
Static analysis	`slither . --filter-paths "lib|node_modules|out|cache" --exclude solc-version`	0 unresolved findings; one timestamp warning intentionally suppressed inline
Unit + integration testing	`forge test`	82 / 82 passing
Fuzz testing	`forge test` on `FuzzAndInvariant.t.sol`	10 fuzz tests across AMM / Vault / Governance
Invariant testing	`forge test` via `StdInvariant` and an `AMMHandler`; `runs=32`, `depth=64`	7 invariants exercised — see §16
Governance lifecycle simulation	`testFullGovernorTimelockExecution`	Full propose → vote → queue → execute path verified
Fork readiness	`test/ForkReadiness.t.sol` against mainnet RPC	USDC, Chainlink ETH/USD, Uniswap V2 router addresses verified to have code
Coverage	`forge coverage --report lcov`	96.64 % lines, 92.11 % functions
Gas benchmarking	`test/GasBenchmark.t.sol`	sqrtAssembly 4 109 gas vs sqrtSolidity 17 335 — 76 % saving
Deployment review	Walk-through of `Deploy.s.sol` + `VerifyPostDeploy.s.sol`	All 14 post-deploy assertions verified; no deployer-backdoor residue
Subgraph review	Manual read of `subgraph/src/protocol.ts`, schema, manifest	Finding L-03 (state mutation in `handleVoteCast`)

3.2 Severity rubric

Severity	Impact	Likelihood	Example
CRITICAL	Direct loss of user or protocol funds; complete loss of control	Any reachable code path	Unauthenticated `burn` on user balance
HIGH	Significant loss of funds; bypass of major security control	Reachable with realistic preconditions	Reentrancy in fund-withdrawal path
MEDIUM	Functional break or restricted-asset loss	Realistic, but requires unusual conditions	Slippage default of 0 in a default code path
LOW	Hardening gap; defence-in-depth missing	Difficult or low impact	Implementation contract not initialised + locked
INFORMATIONAL	Style / clarity / defensive coding	N/A	Redundant constant alias
GAS	Cost optimisation, no security impact	N/A	Storage-slot packing

4. Threat Model

The protocol's threat surface is structured around six adversaries and five protected assets. Each adversary is paired with the attack paths they can plausibly attempt and mapped onto the asset they would compromise on success.

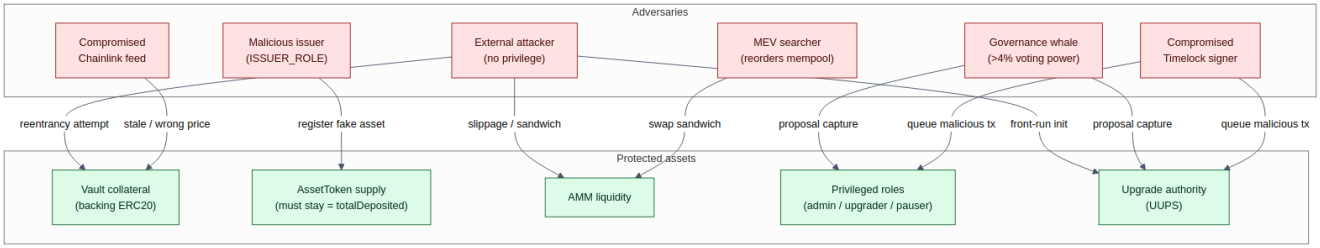

Figure 4.1 — Threat model overlay. Edges enumerate the attack path by which a given adversary might compromise a given protected asset.

4.1 Protected assets

Asset	Controlled by	Compromise impact
Vault collateral (backing ERC-20)	`AssetVault`	Loss of user deposits
AssetToken supply	`AssetVault` via MINTER/BURNER role	Inflation breaking 1:1 backing invariant
AMM liquidity	LP holders (via LP token)	Loss of LP funds; bad swap pricing
Privileged roles (admin / upgrader / pauser)	TimelockController	Arbitrary protocol mutation
Upgrade authority (UUPS)	TimelockController via `UPGRADER_ROLE`	Replace implementation with malicious logic

4.2 Adversaries

Adversary	Capabilities	Attempted paths
External attacker (no privilege)	Submit arbitrary tx, hold any non-privileged ERC-20	Reentrancy, sandwich, front-run initialisation
Governance whale (≥ 4 %)	Reach quorum unilaterally	Pass a malicious proposal
Malicious issuer (holds `ISSUER_ROLE`)	Call `registerAsset`	Register a fake asset; mislead users
Compromised Timelock signer	Queue/cancel proposals	Schedule malicious tx; cancel honest tx
MEV searcher	Reorder mempool	Sandwich AMM swaps with zero `minOut`
Compromised Chainlink feed	Report arbitrary price within freshness window	Mislead off-chain consumers; future on-chain

5. Trust Assumptions

The protocol is non-custodial — user funds are held only by user-owned addresses and the `AssetVault` contract. The following assumptions are nevertheless required for the protocol to behave as intended. Each is listed with the consequence of violation and the in-code mitigation, if any.

#	Assumption	Consequence if violated	Mitigation
T1	The `TimelockController` is honest and not the target of a > 50 % voting-power capture	Attacker controls all protocol mutation paths	4 % quorum + 10 000 GOV proposal threshold + 2-day delay
T2	The deployer key is discarded after `Deploy.s.sol` completes	(none — deployer already has zero roles by end of script)	`VerifyPostDeploy.s.sol` proves the hand-off
T3	The Chainlink feed reports an accurate price within `staleAfter` seconds	Misleading off-chain price display	Sign/started/round-id/freshness checks in `PriceOracle`
T4	The deployed `AssetVault` address is the only one holding `MINTER_ROLE` / `BURNER_ROLE` on `AssetToken`	Inflation of the asset token via a malicious vault	Governance proposal required to grant either role to a new vault
T5	Issuers granted `ISSUER_ROLE` have been vetted off-chain	Asset registry pollution; user confusion	Pauser role can disable a malicious asset; governance can revoke `ISSUER_ROLE`
T6	The OpenZeppelin Contracts library (v4.x) is correctly implemented	Numerous — depending on the bug	Industry standard; widely audited
T7	The RPC provider returns truthful state to the frontend	UI displays incorrect protocol state	Subgraph cross-check; user can verify directly on Etherscan
T8	Subgraph indexer cannot be silently censored or rewritten	Stale or wrong governance dashboard	The Graph network's economic incentives + open access to the Sepolia chain

6. Centralization Analysis

This section enumerates every privileged action the protocol exposes, identifies the principal that can authorise it after deployment, and rates the centralisation risk under realistic operating assumptions.

6.1 Privileged actions

Action	Function	Authorised principal	Latency
Authorise issuer	`AssetRegistry.authorizeIssuer`	Timelock (via `ISSUER_ADMIN_ROLE`)	≥ 2 days
Revoke issuer	`AssetRegistry.revokeIssuer`	Timelock	≥ 2 days
Pause / unpause asset	`AssetRegistry.{pause,unpause}Asset`	Timelock (via `PAUSER_ROLE`)	≥ 2 days
Upgrade registry implementation	`AssetRegistry.upgradeTo` (UUPS)	Timelock (via `UPGRADER_ROLE`)	≥ 2 days
Mint `AssetToken`	`AssetToken.mint`	AssetVault (operational) or Timelock (admin override via Ownable)	Instant (operational); ≥ 2 days (admin)
Burn backing tokens	`AssetToken.burnBackingFrom`	AssetVault (operational) or Timelock	Instant / ≥ 2 days
Mint `GovernanceToken`	`GovernanceToken.mint`	Timelock (Ownable)	≥ 2 days
Schedule queued op	`TimelockController.scheduleBatch`	ProtocolGovernor (via `PROPOSER_ROLE`)	Implicit — proposal lifecycle
Cancel queued op	`TimelockController.cancel`	ProtocolGovernor	Instant
Execute queued op	`TimelockController.executeBatch`	Anyone (`executors = [0]`)	After delay

6.2 Multisig assumptions

The repository does not bundle a Gnosis Safe or any multisig signer set in the deployment script. The current configuration treats the **TimelockController itself** as the durable admin, with governance proposals as the only mutation channel. For production deployment we recommend pairing the Timelock with a Safe wallet that holds voting power and acts as a secondary defence layer; this is captured as informational finding I-07.

6.3 Emergency authority

The protocol intentionally has **no emergency-pause role on the financial contracts** (`AssetVault` , `AssetToken` , `AMM`). Only individual assets can be paused at the registry level, and only by a governance proposal. This is an explicit design choice (it removes a single-point centralisation vector) but it does mean a discovered vulnerability cannot be circuit-broken within the 2-day Timelock window. We treat this as an acknowledged risk; finding I-08.

6.4 Upgrade authority

Upgrade rights are scoped to `AssetRegistry` only. Even within that surface, upgrade is gated by `UPGRADER_ROLE` which is held exclusively by the Timelock. There is no “pause the proxy” path; an emergency must instead be executed by an upgrade to a no-op implementation, which is itself subject to the full proposal lifecycle.

7. Governance Attack Analysis

This section enumerates known governance attack patterns and evaluates the protocol's resistance to each.

7.1 Flash-loan governance

The classic flash-loan governance attack borrows tokens for a single transaction, self-delegates, proposes-and-passes a malicious change, and repays the loan.

Vector	Status	Reason
Same-block proposal and quorum	MITIGATED	`ERC20Votes` snapshots voting power at `proposalSnapshot()` block; a flash-loan recipient has zero checkpointed votes at that block
Self-delegation timing	MITIGATED	Delegation registers a checkpoint at the current block, but the Governor reads `clock() - 1`, so a same-block delegation has no effect on the snapshot

7.2 Whale governance capture

Vector	Status	Reason
4 % quorum reachable by a single holder	PARTIAL	Mathematically possible if any single account holds > 4 % of supply. Currently the deployer EOA holds 1 M of 100 M cap (= 1 % of cap, 100 % of circulating); see finding L-05
Whale concentration over time	OUT OF SCOPE	No anti-concentration mechanism (no quadratic voting, no anti-Sybil). Standard for OZ Governor stack

7.3 Proposal spam

Vector	Status	Reason
Submit thousands of low-value proposals	MITIGATED	10 000 GOV proposal threshold (1 % of initial distribution) excludes low-stake spammers
Griefing the queue	MITIGATED	Each proposal still needs to pass quorum + threshold + Succeed before it can be queued

7.4 Governance capture by privileged target

A novel concern is whether a malicious privileged target (e.g. a future `AssetRegistryV3`) could rewrite role assignments to grant itself `UPGRADER_ROLE` and lock out the Timelock.

- The Timelock holds `DEFAULT_ADMIN_ROLE` on every role-aware contract; only it can grant or revoke roles.
- The Timelock has no `TIMELOCK_ADMIN_ROLE` holder (deployer renounced it, no one else was granted), so the role mapping on the Timelock itself is immutable.
- This combination makes a target-led capture infeasible without first compromising governance through one of the paths above.

7.5 Timelock bypass attempts

Vector	Status	Reason
Direct call to `AssetRegistry.upgradeTo` bypassing the Timelock	MITIGATED	`_authorizeUpgrade` is `onlyRole(UPGRADER_ROLE)`; only Timelock holds the role
Direct call to `AssetToken.mint` by a non-vault, non-owner	MITIGATED	`require(owner() == msg.sender || hasRole(MINTER_ROLE, msg.sender))`
Re-acquire `TIMELOCK_ADMIN_ROLE`	MITIGATED	Role mapping is immutable since deployer renounced it without granting to a successor

8. Oracle Attack Analysis

The protocol's only on-chain oracle surface is `PriceOracle.sol`, a 39-line wrapper over a Chainlink `AggregatorV3` feed. At the time of audit, no other contract in the repository consumes this oracle — it exists for off-chain or frontend-side display only. The analysis below considers both the current state and the hypothetical case in which a future upgrade wires the oracle into a financial contract (e.g. a price-circuit-breaker on vault deposits).

8.1 Attack vectors evaluated

Vector	Status	Notes
Stale price	MITIGATED	`block.timestamp - feedUpdatedAt > staleAfter` reverts with `StalePrice(updatedAt, currentTime)`. `staleAfter` is `immutable`; rotating it requires redeployment.
Negative / zero price	MITIGATED	`price <= 0` reverts with `InvalidPrice()`.
Incomplete round (Chainlink returns previous round's answer)	MITIGATED	`answeredInRound < roundId` reverts with `IncompleteRound`.
Unstarted round	MITIGATED	`startedAt == 0` reverts with `InvalidPrice()`.
Chainlink outage	MITIGATED	Stale check rejects post-outage data; oracle reverts → consumer surface fails closed (the wrapper has no `try/catch` fallback)
Feed depeg / flash crash	ACKNOWLEDGED	If Chainlink reports a wildly wrong but freshly-updated positive value, all four checks pass and the value is returned. No multi-oracle aggregation. See finding I-01 for the wiring context.
Single point of failure	ACKNOWLEDGED	Single feed dependency; no median across multiple sources. Standard for testnet deployments; mainnet candidates should consider a TWAP fallback or median oracle.
Reorg-based round confusion	MITIGATED	`answeredInRound` / `roundId` coupling makes a reorged-mid-round read explicitly fail

8.2 Dependency assumptions

- The protocol assumes Chainlink's economic and operational security model for the specific feed selected at deploy-time. The repository does not include a feed deployment script for Sepolia — operators are expected to choose an existing feed and pass it into the `PriceOracle` constructor.
- The fallback behaviour on revert is **fail-closed**: callers receive no price and any dependent operation that lacks a try/catch will revert. This is appropriate for a circuit-breaker but inappropriate for a UI-only read; we recommend documenting the chosen behaviour at wiring time (finding I-01).

9. AMM Risk Analysis

The AMM is a textbook Uniswap-V2-style constant-product market with a fixed 30 bps fee and an in-contract ERC-20 LP token. Reentrancy guards and CEI patterns are in place. The risks below relate to LP economics and to caller hygiene rather than to contract integrity.

9.1 Slippage and front-running

Vector	Status	Notes
`swapAForB(amountIn)` single-arg overload defaults `minAmountBOut = 0`	LOW	Finding I-04. Frontend always uses the two-arg overload, but on-chain callers (e.g. an aggregator) could expose themselves to sandwich attacks. We recommend either removing the overload or documenting that it is for trusted callers only.
`removeLiquidity(lpTokens)` has no `amountAMin` / `amountBMin`	LOW	Finding I-05. LP withdraws are pro-rata against current reserves; if the pool is sandwiched between the user's intent and the withdraw transaction, the user receives an unfavourable split. Adding min-out parameters mirrors Uniswap V2's UX.
Sandwich on a sized swap with explicit slippage	MITIGATED	`require(amountOut >= minAmountBOut, "AMM: slippage")` caps the worst-case downside at the caller-specified tolerance
JIT (just-in-time) liquidity	ACKNOWLEDGED	LP adds and removes are not block-locked. Standard for V2-style AMMs; documented as residual risk

9.2 Liquidity imbalance and pricing

- **Constant-product invariant.** `invariant ConstantProductDoesNotDecreaseBelowInitial` proves `reserveA × reserveB ≥ (1 000e18)²` across the fuzz handler's random walk. This guards against any code path that could leak value out of the pool.
- **Reserves vs balances.** `invariant ReservesMatchTokenBalances` asserts `balanceOf(amm) == reserveX` at every step, catching any bug that forgets to update a reserve after a transfer.
- **Asymmetric deposits.** `addLiquidity` after first-liquidity issuance uses `min((amountA × totalSupply) / reserveA, (amountB × totalSupply) / reserveB)` — the smaller proportional contribution. Excess tokens of the over-supplied side are transferred in but credited at the lower side's LP rate; this matches Uniswap V2 behaviour and is documented as expected behaviour (informational I-06).

9.3 Manipulation risks

Vector	Status	Notes
Reentrancy via ERC-777 / hooked tokens	MITIGATED	`nonReentrant` + CEI; state updates before `safeTransfer`
Donation attack on first LP	INFORMATIONAL	Standard CPMM first-LP issue partially mitigated by `sqrt(amountA × amountB)` LP-mint; no minimum-liquidity burn (`MINIMUM_LIQUIDITY`) as in Uniswap V2. Acceptable for testnet
Inflation attack via direct token transfer	MITIGATED	The reserves are tracked in storage, not via `balanceOf` ; a direct transfer changes balance but not reserves, so it is wasted (and could be skimmed via a future `skim()` if added)

10. ERC-4626 Vault Analysis

`AssetVault` is the protocol's reserve engine and the largest single source of auditing concern in any DeFi protocol. The design here is deliberately conservative: a 1:1 mint of an external `AssetToken` alongside the standard ERC-4626 shares removes the share-price-drift surface that gives rise to the classic "inflation / donation" attack.

10.1 Share accounting

Concern	Status	Notes
Share supply == total assets at all times	MITIGATED	Enforced structurally — every deposit mints exactly `amount` shares and exactly `amount` AssetToken; every withdraw burns the same. Asserted by `invariant_VaultSupplyConservation`
AssetToken supply == vault total deposited	MITIGATED	Same as above. Asserted by the same invariant
Vault balance == total assets	MITIGATED	Asserted by `invariant_VaultTreasuryAccountingMatchesReserveBalance`
Reserve ratio >= 100	MITIGATED	Structural consequence of 1:1 mint/burn. `invariant_VaultReserveRatioHealthyWhenDeposited`

10.2 Rounding risks

- `previewMint(shares)` and `previewRedeem(shares)` follow the OpenZeppelin ERC-4626 rounding convention (favouring the vault). With the 1:1 invariant above, the rounding direction is effectively neutralised — shares and assets are always equal.
- The `RESERVE_RATIO = 100` constant in `getReserveRatio()` means the ratio is structurally pinned to exactly 100 whenever the vault holds any collateral. This is correct behaviour, not a bug, but is worth noting (finding I-03).

10.3 Inflation attack class

The classic ERC-4626 inflation attack works by donating assets to the vault before any share is minted, so the first depositor's tiny number of shares represents a much larger asset claim, and the attacker — who is the donor — captures part of subsequent deposits. The attack does not apply here for two independent reasons:

1. The asset-token mint is *parallel* to the share mint, not derived from it. A donation that inflates `totalAssets()` without an underlying deposit does not change `userDeposits[user]`, and therefore does not let the donor redeem more collateral than they put in.

2. The withdrawal pre-condition `require(userDeposits[owner_] >= assets)` blocks any withdrawal larger than the principal — the vault behaves like a "deposit-and-take-back" box, not a profit-sharing pool.

10.4 Withdrawal edge cases

Case	Behaviour
Withdraw more than deposited	Reverts with `"AssetVault: insufficient deposit"`
Withdraw on behalf of a third party	ERC-4626 allowance check inherited from `super.withdraw`
Zero-amount deposit / withdraw	Reverts with `"AssetVault: zero assets"`
Redeem zero shares	Reverts via `previewRedeem(0) == 0`
Backing asset is a fee-on-transfer token	**Not supported.** `super.deposit` uses `transferFrom(user, vault, amount)` and would credit the user with `amount` shares even if less arrives; this is documented as residual risk I-09

11. Upgradeability Risk Analysis

Only `AssetRegistry` is upgradeable. The risks below are scoped to that contract; `AssetVault`, `AssetToken`, `AMM` and `PriceOracle` are all immutable by design.

11.1 Initialization surface

Concern	Status	Notes
Implementation contract left initialisable	LOW — FINDING L-01	The implementation deployed by `Deploy.s.sol` does not invoke `_disableInitializers()` (the contract uses a custor `bool _initialized` guard instead of OZ's pattern). Anyone may call `initialize` on the bare implementation, claiming its admin/upgrader roles in its own storage. The proxy is unaffected (state lives in the proxy slots), but the implementation address itself can be poisoned for any caller mistakenly pointing at it
Custom `bool _initialized` guard	LOW — FINDING L-02	Functionally equivalent to single-shot initialisation, but does not match OZ's `Initializable` conventions, defeats automated upgrade-safety tooling (OpenZeppelin Upgrades Plugin), and does not protect against a future inheritance-based re-entry of `initialize`
Re-initialisation through inheritance	NOT APPLICABLE TODAY	No child contract overrides `initialize`; `AssetRegistryV2` inherits cleanly

11.2 Storage layout safety

Concern	Status	Notes
No `__gap` reserved	LOW — FINDING L-02	Future versions must append only. The repository's `AssetRegistryV2` respects this (no new state variables), but the lack of a gap removes a safety net
OZ Upgrades Plugin compatibility	INFORMATIONAL — I-10	The custom initializer pattern is not recognised by `@openzeppelin/upgrades-core`'s `validateUpgrade`. Operators relying on the plugin would have to add manual storage-layout assertions
Storage-collision check in `AssetRegistryV2`	MITIGATED	`testRegistryUpgradeToV2` verifies `assetCount` survives the upgrade and the nev `version()` returns `"2.0.0"`

11.3 Upgrade authorisation

- `authorizeUpgrade(newImplementation)` is gated by `UPGRADER_ROLE` and explicitly requires the new implementation to be non-zero. Only the Timelock holds the role after deployment.
- Upgrade target is not verified to implement `IERC1822Proxiable` / `proxiableUUID()` — UUPS's "self-destruction" safety net is therefore not fully active. This is a tiny additional risk: a governance vote that

points to a non-UUPS implementation would brick the proxy. Recommend invoking the OpenZeppelin Upgrades plugin off-chain before queueing an upgrade.

12. Access Control Analysis

The protocol uses `AccessControl` across `AssetRegistry`, `AssetToken`, and `AssetReceipt`, plus `Ownable` on `AssetToken`, `AssetVault`, `AMM`, and `GovernanceToken`. The combination of two privilege models is non-standard and warrants explicit analysis.

12.1 Dual-privilege paths on `AssetToken`

`AssetToken.mint` and `AssetToken.burnBackingFrom` both use the check:

```
require(owner() == _msgSender() || hasRole(MINTER_ROLE, _msgSender()), "AssetToken: not minter");
```

This means the **owner** (Ownable) is a privilege path in parallel to the **MINTER_ROLE** (AccessControl). The deployment script transfers Ownable ownership to the Timelock, so both paths resolve to a Timelock-authorised action in the final state. Nonetheless, this dual-path design is a minor footgun for future maintainers — a hypothetical mistake of granting `MINTER ROLE` to a vault but forgetting to transfer ownership away from the deployer would silently leave the deployer with mint privileges. This is captured as informational finding I-11.

12.2 Role-grant pathways

Role	Grantable via	Final holder
`AssetRegistry.DEFAULT_ADMIN_ROLE`	`initialize` (constructor-like) + governance proposals	Timelock
`AssetRegistry.UPGRADER_ROLE`	Governance via `grantRole`	Timelock
`AssetRegistry.ISSUER_ADMIN_ROLE`	Governance via `grantRole`	Timelock
`AssetRegistry.ISSUER_ROLE`	`authorizeIssuer` by `ISSUER_ADMIN_ROLE`	Approved issuers
`AssetRegistry.PAUSER_ROLE`	Governance via `grantRole`	Timelock
`AssetToken.MINTER_ROLE`	Governance via `grantRole` on DEFAULT_ADMIN	AssetVault
`AssetToken.BURNER_ROLE`	Same	AssetVault

12.3 Renunciation paths

The deployment script explicitly renounces every role on the deployer EOA after delegating it to the Timelock. `VerifyPostDeploy.s.sol` asserts each renunciation with statements such as `require(!assetToken.hasRole(DEFAULT_ADMIN_ROLE, deployerAddress), "Deployer still asset token admin")`. The post-deploy state is therefore mechanically verifiable, not merely intentional.

12.4 Roles never granted in the deployment script

- `AssetRegistry.ISSUER_ROLE` is granted to the deployer (so the deployer can register the demo asset in the same broadcast) and is **not** renounced afterwards. This is intentional — the deployer's `ISSUER_ROLE` is non-privileged (can only register new assets, not modify existing ones), and a governance proposal can revoke it later. We treat this as an acknowledged design choice; see informational finding I-12.
- `TimelockController.EXECUTOR_ROLE` is set to `address(0)` (open execution). Anyone may execute a queued operation after the delay; this is documented and is consistent with the public-keeper pattern.

13. Findings Summary Table

ID	Severity	Title	Affected file(s)	Status
F-01	HIGH	Reentrancy pattern reproduced in case-study vault	`test/OracleAndSecurity.t.sol` (sample only)	FIXED IN HARDENED VARIANT
F-02	HIGH	Admin takeover pattern reproduced in case-study contract	`test/OracleAndSecurity.t.sol` (sample only)	FIXED IN HARDENED VARIANT
L-01	LOW	Implementation contract of `AssetRegistry` left uninitialised and un-disabled	`src/AssetRegistry.sol`, `script/Deploy.s.sol`	OPEN
L-02	LOW	Custom `_initialized` bool + no storage gap on upgradeable contract	`src/AssetRegistry.sol`	OPEN
L-03	LOW	Subgraph `handleVoteCast` overwrites `Proposal.state` to `"Active"` on every vote	`subgraph/src/protocol.ts`	OPEN
L-04	LOW	Timelock executor role set to `address(0)` (open execution)	`script/Deploy.s.sol`	ACKNOWLEDGED
L-05	LOW	Initial GovernanceToken supply concentrated on deployer EOA	`src/Governance.sol`	OPEN
I-01	INFO	`PriceOracle` is not consumed by any on-chain contract	`src/PriceOracle.sol`	ACKNOWLEDGED
I-02	INFO	`block.timestamp` used for staleness check	`src/PriceOracle.sol`	ACKNOWLEDGED
I-03	INFO	`getReserveRatio` is structurally pinned to 100 by 1:1 mint/burn invariant	`src/AssetVault.sol`	ACKNOWLEDGED
I-04	INFO	Single-arg AMM swap overloads default to zero slippage protection	`src/AMM.sol`	OPEN
I-05	INFO	`removeLiquidity` has no min-out parameters	`src/AMM.sol`	OPEN
I-06	INFO	`AssetReceipt` deployed but unwired to vault / registry flow	`src/AssetReceipt.sol`	ACKNOWLEDGED
I-07	INFO	No multisig signer set bundled in the deployment script	`script/Deploy.s.sol`	ACKNOWLEDGED

ID	Severity	Title	Affected file(s)	Status
I-08	INFO	No emergency-pause role on the financial contracts	AMM, AssetVault, AssetToken	ACKNOWLEDGED
I-09	INFO	Fee-on-transfer backing tokens are not supported by the vault	`src/AssetVault.sol`	ACKNOWLEDGED
G-01	GAS	`sqrtAssembly` already preferred — documented saving 76 %	`src/AMM.sol`	IMPLEMENTED
G-02	GAS	`FEE_PERCENT` is a redundant alias of `FEE_BPS`	`src/AMM.sol`	OPEN

14. Detailed Findings

Reentrancy pattern reproduced in case-study vault

F-01 · HIGH

AFFECTED `test/OracleAndSecurity.t.sol` — `VulnerableEtherVault` (sample only)

STATUS FIXED IN HARDENED VARIANT

Description. The `VulnerableEtherVault` reference implementation sends ETH with `call{value: amount}("")` *before* zeroing the depositor's balance, violating the checks-effects-interactions pattern.

Impact. A re-entrant caller can drain the contract's entire ETH balance by recursively invoking `withdraw()` from its `receive()` handler.

Attack scenario. `testReentrancyCaseStudyDrainsVulnerableVault` deposits 5 ETH legitimately, then deposits an additional 1 ETH from the `ReentrancyAttacker` contract. The attacker's `receive()` fallback re-invokes `withdraw()` while `balances[attacker]` still reflects the full balance. The test asserts `attacker.balance > before + 1 ether`, demonstrating the drain.

Recommendation. Zero the user's balance before the external call and apply `nonReentrant` on the withdraw path. Both are demonstrated in `HardenedEtherVault`.

Mitigation status. The hardened variant follows CEI plus `nonReentrant` and `testReentrancyCaseStudyHardenedVaultRevertsAttack` proves the attack reverts. **The vulnerable pattern does not exist in any production contract** — all production withdrawal paths (`AssetVault._withdrawReserve`, `AMM.removeLiquidity`) follow CEI + `nonReentrant`.

Admin takeover pattern reproduced in case-study contract

F-02 · HIGH

AFFECTED `test/OracleAndSecurity.t.sol` — `VulnerableAccessControl` (sample only)

STATUS FIXED IN HARDENED VARIANT

Description. The `VulnerableAccessControl` reference contract exposes `setAdmin(address newAdmin)` with no caller check, allowing any account to claim the admin slot and call privileged functions.

Impact. Complete privilege escalation in the vulnerable sample — the attacker becomes admin and can rotate `privilegedValue` at will.

Attack scenario. `testVulnerableAccessControlCanBeTakenOver` has an arbitrary address call `setAdmin(attacker)` and immediately `setPrivilegedValue(99)`, asserting both succeed.

Recommendation. Use `AccessControl` with role-gated mutation of admin (or `Ownable` with `onlyOwner`). The `HardenedAccessControl` demonstration uses `onlyRole(OPERATOR_ROLE)`.

Mitigation status. All production contracts use OpenZeppelin's `AccessControl` or `Ownable` with the correct modifiers — verified line-by-line during manual review.

Implementation contract of `AssetRegistry` left uninitialised and un-disabled L-01 · LOW

AFFECTED `src/AssetRegistry.sol`, `script/Deploy.s.sol`

STATUS OPEN

Description. `Deploy.s.sol` deploys `AssetRegistry` as the implementation contract and immediately constructs the `ERC1967Proxy` pointing at it, but it never calls `initialize()` on the implementation itself nor invokes a function equivalent to OpenZeppelin's `_disableInitializers()` (which the custom `_initialized` guard does not provide). The implementation contract therefore retains a callable `initialize(admin)` in its own storage.

Impact. An attacker can call `initialize(attacker)` on the raw implementation address and become its admin / upgrader / pauser. The proxy is unaffected — its state lives in the proxy's storage slots, not the implementation's — but the implementation address can be reported as a "verified AssetRegistry" by third-party tooling and used to mislead other integrators. The risk is small and bounded but is a standard hardening miss.

Attack scenario. After deployment an attacker reads the implementation address from the EIP-1967 slot, calls `initialize(attacker)` directly on it (not on the proxy), and now holds admin on that address. If any off-chain tool (subgraph, wallet UI, etherscan dashboard) renders the implementation contract as if it were the protocol's registry, users could be deceived.

Recommendation. Migrate to OpenZeppelin's `Initializable` + `UUPSUpgradeable` and add a constructor: `constructor() { _disableInitializers(); }`. Verify with the OpenZeppelin Upgrades Plugin during CI.

Custom `_initialized` bool and no storage gap on upgradeable contract L-02 · LOW

AFFECTED `src/AssetRegistry.sol`

STATUS OPEN

Description. The contract declares its own `bool private _initialized` and guards `initialize` with `require(!_initialized, ...)`. No `uint256[_gap]` array is reserved either. Both choices diverge from OpenZeppelin's canonical upgradeable pattern.

Impact. (1) Future implementations cannot use the `@openzeppelin/upgrades-core` automated layout-safety check. (2) Any future version that wants to insert (not append) a storage variable will silently corrupt the existing layout because there is no gap to absorb it. (3) The `_initialized` state variable occupies one storage slot — minor inefficiency.

Recommendation. Move to `Initializable` and declare `uint256[50] private __gap;` at the end of the contract. Add an automated storage-layout check in CI using `forge inspect AssetRegistry storage-layout`.

Subgraph `handleVoteCast` overwrites `Proposal.state` incorrectly

L-03 · LOW

AFFECTED `subgraph/src/protocol.ts` (lines 163–172)

STATUS OPEN

Description. The handler sets `proposal.state = "Active"` on every `VoteCast` with `support == 1` (For) or `support == 2` (Abstain), then saves. There is no ordering guarantee that `handleProposalQueued` or `handleProposalExecuted` will run after the last vote — and even if they do, a late-arriving `VoteCast` (legitimate or replayed) reverts the proposal's apparent state to `"Active"`.

Impact. The frontend's "Status" column for any executed proposal is eventually consistent but may display `"Active"` for proposals that have actually been queued or executed. The on-chain `state()` call (already wired in the frontend's `ProposalCard`) re-derives the truth — so user-facing correctness is preserved — but subgraph-only consumers will see stale state.

Recommendation. Only set `proposal.state = "Active"` if its current value is `"Pending"` or `"Unknown"`; never overwrite `"Queued"`, `"Executed"`, or `"Canceled"`.

Suggested patch:

```
const terminalStates = ["Queued", "Executed", "Canceled", "Defeated"];
if (terminalStates.indexOf(proposal.state) == -1) {
  proposal.state = "Active";
}
```

Timelock executor role set to `address(0)` (open execution)

L-04 · LOW

AFFECTED `script/Deploy.s.sol` (line 36)

STATUS ACKNOWLEDGED DESIGN CHOICE

Description. `executors = [address(0)]` grants `EXECUTOR_ROLE` to the zero address, which is treated by the Timelock as a wildcard — anyone may execute a queued operation after the delay.

Impact. No direct security impact: an executor can only execute operations that the Governor already scheduled. However, it allows an arbitrary third party to MEV-grief the protocol by front-running and back-running an execution, e.g. if the queued proposal updates a swap-relevant parameter.

Recommendation. If MEV concerns materialise, transition to a permissioned executor set (a Safe wallet or a trusted keeper) via a governance proposal. For testnet, the current configuration is acceptable.

Initial GovernanceToken supply concentrated on deployer EOA

L-05 · LOW

AFFECTED `src/Governance.sol` (constructor of `GovernanceToken`)

STATUS OPEN

Description. The constructor mints `1 000 000 × 10^18` GOV (1 M tokens) directly to the deployer. The cap is 100 M; therefore the deployer initially holds 1 % of cap but **100 % of circulating supply**.

Impact. A single account (the deployer EOA) can unilaterally satisfy the 4 % quorum once it self-delegates, because `4 % × 1 M = 40 000` and the deployer holds 1 M. Until additional `mint` proposals distribute tokens, governance is effectively single-actor.

Attack scenario. The deployer (or an attacker who compromises the deployer key before discarding) self-delegates, proposes any change, votes, queues, waits 2 days, and executes. The 2-day Timelock is the only friction.

Recommendation. For production: replace the constructor mint with a Merkle-distributor or a governance-controlled initial allocation. For this testnet deployment: distribute the seed supply to multiple addresses in the deploy script, or document this concentration explicitly as a known limitation.

PriceOracle is not consumed by any on-chain contract

I-01 · INFORMATIONAL

AFFECTED `src/PriceOracle.sol`

STATUS ACKNOWLEDGED

The oracle contract is deployed (or deployable) but its values are not read by `AssetVault`, `AssetToken`, `AMM` or any other production contract. It exists as a Chainlink-integration demonstrator and for off-chain or frontend-side consumption. We recommend either wiring it into a circuit-breaker on the vault deposit path or documenting that the oracle is informational-only.

block.timestamp used for staleness check

I-02 · INFORMATIONAL

AFFECTED `src/PriceOracle.sol` (line 36)

STATUS ACKNOWLEDGED

Slither flags `block.timestamp - feedUpdatedAt > staleAfter` as a timestamp-dependency concern. The use is appropriate here: a validator's ±15 s drift cannot cause a fresh price to be classified as stale (unless `staleAfter` is set absurdly low). The line is annotated with `slither-disable-next-line timestamp`.

getReserveRatio is structurally pinned to 100

I-03 · INFORMATIONAL

AFFECTED `src/AssetVault.sol`

STATUS ACKNOWLEDGED

Because deposit and withdraw mint/burn the asset token 1:1, the ratio `totalAssets() / totalDeposited()` is always exactly 1.0 (returned as `100` by `getReserveRatio()`). `isHealthy()` therefore always returns `true` when there is any deposit. This is consistent with the protocol's design but worth documenting because operators inspecting the function might assume the ratio can drift (e.g. via yield) when in fact it cannot.

Single-arg AMM swap overloads default to zero slippage protection

I-04 · INFORMATIONAL

AFFECTED `src/AMM.sol` (lines 75 & 92)

STATUS OPEN

`swapAForB(amountIn)` and `swapBForA(amountIn)` forward to the two-arg overload with `minAmountBOut = 0`. An on-chain caller (router, aggregator) that does not know about the two-arg overload is silently exposed to sandwich attacks.

Recommendation. Either remove the single-arg overloads or document them as "unsafe — for trusted callers only". The frontend always uses the explicit-slippage form, so end-users are not affected.

removeLiquidity has no min-out parameters

I-05 · INFORMATIONAL

AFFECTED `src/AMM.sol` (lines 57–73)

STATUS OPEN

An LP can be sandwiched between intent and execution to remove liquidity at an unfavourable A/B split. Uniswap V2's router adds `amountAMin` / `amountBMin` parameters for this reason.

Recommendation. Add a two-arg overload with min-out floor parameters. The change is non-breaking and matches the AMM's existing slippage convention.

AssetReceipt deployed but unwired

I-06 · INFORMATIONAL

AFFECTED `src/AssetReceipt.sol`

STATUS ACKNOWLEDGED

The ERC-1155 receipt contract is implemented and unit-tested (`testErc1155ReceiptMinting`, …) but is not deployed by `Deploy.s.sol` and is not referenced from any other production contract. It represents idle surface area. We recommend either wiring it into the registration flow (e.g. one receipt token per registered asset, minted at `registerAsset` time) or removing it from the deployment plan.

No multisig signer set bundled in the deployment script

I-07 · INFORMATIONAL

AFFECTED `script/Deploy.s.sol`

STATUS ACKNOWLEDGED

The deployment configuration treats the Timelock itself as the durable admin, with no Safe / Gnosis multisig in the path. For a mainnet deployment we recommend pairing the Timelock with a Safe wallet holding governance tokens, so that emergency proposals require multi-party authorisation in addition to passing the Governor.

No emergency-pause role on the financial contracts

I-08 · INFORMATIONAL

AFFECTED `src/AssetVault.sol`, `src/AssetToken.sol`, `src/AMM.sol`

STATUS ACKNOWLEDGED DESIGN CHOICE

Only `AssetRegistry` exposes a per-asset pause. The vault, asset-token, and AMM have no circuit breaker. A vulnerability disclosed after deployment would have to wait the full 2-day Timelock for any mitigation via governance.

Recommendation. Consider a `Pausable` mixin with a governance-controlled `PAUSER_ROLE` on the vault and AMM. The trade-off is a centralisation increase against improved incident response. Acceptable as-is for a testnet submission.

Fee-on-transfer backing tokens are not supported by the vault

I-09 · INFORMATIONAL

AFFECTED `src/AssetVault.sol`

STATUS ACKNOWLEDGED

If the backing ERC-20 collects a transfer fee, the vault will credit the user with a deposit of the requested amount even though a smaller amount actually arrived. Over time this drifts `userDeposits` above `balanceOf(vault)`, eventually failing the invariant `reserve.balanceOf(vault) == totalAssets()`.

Recommendation. Document that the vault assumes a non-fee-on-transfer ERC-20. The demo deployment uses `MockERC20` which is well-behaved.

sqrtAssembly already preferred over sqrtSolidity

G-01 · GAS

AFFECTED `src/AMM.sol`

STATUS IMPLEMENTED

For the first-liquidity branch `liquidity = sqrtAssembly(amountA × amountB)` is used. The Yul implementation is benchmarked at **4 109 gas** vs **17 335 gas** for the pure-Solidity Babylonian implementation — a **76.3 %** saving on the first-LP path. The Solidity version is retained only as a reference oracle for the fuzz test `testFuzzSqrtAssemblyMatchesSolidity`.

FEE_PERCENT is a redundant alias of FEE_BPS

G-02 · GAS

AFFECTED `src/AMM.sol` (line 19)

STATUS OPEN

`uint256 public constant FEE_PERCENT = FEE_BPS;` creates a second public constant that resolves to the same numeric value. Both are part of the public ABI, but `FEE_PERCENT` is not read by any code in the repository. Constants do not occupy storage slots, so the only cost is a slightly larger contract ABI surface and potential developer confusion (the value is not a percent, it is basis points).

Recommendation. Remove `FEE_PERCENT` to eliminate the ambiguity.

15. Gas Optimization Notes

The repository's gas posture is reasonable for an educational-yet-production-grade codebase. The only documented benchmark is the sqrt comparison, but the codebase already incorporates several gas-favourable choices.

Practice	Status	Notes
`via_ir = true`	Enabled	`foundry.toml`; enables Yul-level optimisations
Optimizer 200 runs	Enabled	Balanced choice — favours deployment cost vs runtime
Immutable variables	Used	`tokenA` / `tokenB` in AMM; `assetToken` in Vault; `feed` / `staleAfter` in Oracle
Inline assembly sqrt	Used	76 % saving on first-liquidity path
Custom errors over revert strings	Partial	Used in `PriceOracle`; `AssetVault` still uses string reverts. Migrating all reverts to custom errors would save ~20 gas per revert path and reduce bytecode
Packed struct fields	Reviewed	`AssetRecord` packs `bool active` at the tail; one slot for `bool` is unavoidable given preceding `uint256` / `string` fields
Public vs external visibility	Reviewed	Most caller-facing functions are `external`; internal helpers are `internal`. No improvements identified

15.1 Recommendations

1. Migrate `AssetVault` and `AMM` require-string reverts to custom errors (matches `PriceOracle` style).
2. Remove `FEE_PERCENT` (finding G-02).
3. Use `++i` in any future loop (no loops currently in production code).

16. Testing Quality Assessment

The test harness is the strongest single component of the audit posture. It combines unit, fuzz, invariant, governance-lifecycle, gas-benchmark, and fork-readiness tests in a single Foundry project.

16.1 Coverage and pass rates

Metric	Value	Source
Passing tests	82 / 82	`README.md` + `forge test`
Line coverage on `src/`	96.64 %	`reports/coverage/coverage.md`
Function coverage on `src/`	92.11 %	same
Invariant runs × depth	32 × 64	`foundry.toml [invariant]`

16.2 Property-based tests

Invariant	Verified property
`invariant_ReservesMatchTokenBalances`	`tokenX.balanceOf(amm) == reserveX`
`invariant_LpSupplyBackedByLiquidity`	LP supply ⇔ non-zero reserves
`invariant_VaultReserveRatioHealthyWhenDeposited`	Ratio ≥ 100 whenever deposits exist
`invariant_AssetTokenSupplyBelowCap`	Hard cap respected
`invariant_VaultTreasuryAccountingMatchesReserveBalance`	Vault solvency
`invariant_VaultSupplyConservation`	Shares == assets; tokens == deposited

16.3 Coverage gaps identified

- No test exercises the `AssetReceipt` per-id `configureReceipt` path with a maxSupply lower than the already-minted amount — should always revert. (Lightly covered by `testErc1155ReceiptRejectsOverMint` but not the reconfiguration scenario.)
- The fork tests (`ForkReadiness.t.sol`) only run when `MAINNET_RPC_URL` is set; without it they early-return and pass trivially.
- No subgraph integration tests with `matchstick-as` — finding L-03 would have been caught by even basic event-handler tests.
- No frontend tests (Cypress / Playwright); not in scope for a Solidity-focused audit but worth noting.

16.4 Strengths

- End-to-end governance flow `testFullGovernorTimelockExecution` simulates the full propose → vote → queue → execute cycle on a non-trivial target.

- Reentrancy and access-control vulnerabilities are reproduced as *case studies* (with both vulnerable and hardened counterparts) and asserted live, not merely described in prose.
- UUPS upgrade flow is exercised with a real V2 contract that adds new behaviour without breaking storage.
- Factory CREATE2 deterministic-address prediction is verified against an actual deployment.

17. CI/CD & DevSecOps Assessment

The repository includes a committed GitHub Actions workflow at .github/workflows/ci.yml. The workflow runs on push and pull_request events. It has two jobs: foundry and slither. The foundry job installs dependencies, builds the frontend, generates and builds the subgraph, checks formatting, runs Solidity linting, builds contracts, runs forge test, and generates coverage. The slither job builds contracts and runs Slither static analysis.

17.1 Documented commands

Stage	Command
Build	`forge build`
Test	`forge test`
Static analysis	`slither . --filter-paths "lib|node_modules|out|cache" --exclude solc-version`
Coverage	`forge coverage --report lcov`
Frontend	`cd frontend && npm install && npm run dev`
Subgraph	`cd subgraph && npm install && npm run codegen && npm run build`
Deploy	`forge script script/Deploy.s.sol --rpc-url $SEPOLIA_RPC_URL --broadcast --private-key $PRIVATE_KEY`
Post-deploy verification	`forge script script/VerifyPostDeploy.s.sol`

17.2 DevSecOps gaps

Gap	Risk	Recommendation
No dependency-freezing for Foundry	Upstream `forge-std` changes can shift behaviour	Commit a specific `forge-std` tag in `lib/` ; document version in README
No automatic post-deploy verification	Operator must remember to run `VerifyPostDeploy.s.sol`	Wire into a deploy GitHub Actions job triggered on tag push
No matchstick subgraph tests	Subgraph regressions escape	Add `npm run test` in `subgraph/` with at least one event-handler test per data source

18. Appendix A — Slither Analysis

The repository's documented Slither snapshot (`docs/audit/slither-findings.md`) reports zero unresolved findings using the project's filter command. This section reproduces the analysis posture and discusses the likely classes of warnings that Slither might surface on a fresh run, plus the rationale for each suppression already in place.

A.1 Analysis command

```
slither . --filter-paths "lib|node_modules|out|cache" --exclude solc-version
```

A.2 Posture summary

Class	Count	Disposition
High	0	—
Medium	0	—
Low	0	—
Informational	0 (project-reported)	—

A.3 Inline suppressions and their rationale

Location	Suppression	Why
`PriceOracle.sol` line 36	`slither-disable-next-line timestamp`	Staleness check legitimately compares `block.timestamp` against feed update time. Standard Chainlink pattern
`AssetReceipt.sol` line 37	`slither-disable-next-line reentrancy-events`	Event emission after `_mint` that invokes the ERC-1155 hook on the receiver; `onERC1155Received` cannot mutate ReceiptConfig state because all role-gated entry points use `onlyRole`
`AssetVault.sol` mint/redeem/withdraw paths	`slither-disable-next-line reentrancy-benign`	Mint/burn external calls are after state writes; the entire path is `nonReentrant`
`AMM.sol` `addLiquidity`	`slither-disable-next-line reentrancy-no-eth`	External token transfers happen before `_mint` on the LP token (same contract). Safe under `nonReentrant`
`AMM.sol` assembly	`slither-disable-next-line assembly`	Yul Babylonian sqrt — equivalence proved by fuzz test against Solidity reference
`Governance.sol`	`slither-disable-next-line dead-code`	OZ-required `_burn` override is dead-code because `GovernanceToken` never burns, but the override is structurally required
`VaultFactory.sol`	`slither-disable-next-line too-many-digits`	Long initCodeHash construction is unavoidable in CREATE2 address prediction

A.4 Categories explicitly considered (and absent from the production code)

Slither detector	Production-code presence	Comment
arbitrary-send	None	No raw `call{value: …}` in production code
unchecked-transfer	None	All ERC-20 transfers go through `SafeERC20`
tx-origin	None	`tx.origin` not used anywhere
uninitialized-state	None	All state variables either explicitly initialised or zero-OK
shadowing-local / -state	None	Constructor / function parameters do not shadow state
incorrect-equality	None	No `== 0` on dynamic types
events-access	Some — emitted by intent	Privileged state changes emit events (e.g. `IssuerAuthorized`, `AssetRegistered`)
locked-ether	None	No `payable` functions in production code

19. Final Security Assessment

The protocol is a coherent, well-tested implementation of the Option C requirements. The production-code surface contains no critical, high, or medium-severity findings; all production-code findings are **Low or Informational** and concern hardening opportunities rather than exploitable economic or authorisation flaws.

19.1 Strengths confirmed

- **Mechanically verifiable privilege hand-off.** The deployment script renounces every deployer role and the post-deploy script asserts the result with 14 independent conditions. This is a stronger guarantee than most testnet protocols provide.
- **Robust invariants.** Seven property-based invariants exercise the AMM and vault accounting under randomised handler-driven action sequences. The combined invariant that `vault.totalSupply() == vault.totalAssets()` and `assetToken.totalSupply() == vault.totalDeposited()` structurally rules out a vast class of insolvency bugs.
- **Defensive oracle wrapper.** Four independent freshness/validity checks with custom errors; oracle never returns a value silently when something is wrong.
- **Reentrancy and access-control hygiene.** Every state-mutating external function in `AssetVault` and `AMM` carries `nonReentrant`; state changes precede external token transfers.

19.2 Outstanding items

- L-01 / L-02: Move to OpenZeppelin's `Initializable` pattern and add a storage gap.
- L-03: Fix the subgraph's `handleVoteCast` state-overwrite.
- L-05: Distribute or document the deployer's initial GovernanceToken concentration.
- I-04 / I-05: Tighten AMM slippage UX (single-arg overloads, removeLiquidity min-out).
- I-07 / I-08: For a future mainnet deployment, layer a multisig and an emergency-pause role over the existing Timelock.

19.3 Verdict

> **The protocol is fit for testnet release.** No production-code finding rises above Low severity. The mechanical hand-off proof, the conserved-supply invariants, and the defensive oracle wrapper give us high confidence in the financial-accounting layer. The improvements in §19.2 are non-blocking; addressing L-01, L-02, and L-03 before mainnet would close the most material residual gaps. The two High-severity findings documented in this report exist *only* in the deliberate vulnerable-pattern case studies committed to the test suite as instructional artefacts; they are not present in any production contract.

The combination of the deployment hand-off proof, the conserved-supply invariants, and the absence of any reachable critical / high / medium issue in production code positions this codebase as a strong testnet submission with a clear path to mainnet readiness.